

Filter Overview

Filters are placed on the connections between a route's input and output communication points. Their purpose is to control and direct message processing, based on the organization's business requirements.

Testing to ensure that the required business logic has been applied correctly can occur within the filter itself, without checking in the route. The testing configuration, along with any test messages, can be included in the RLC for the configuration.

Filter Functionality

Filters can perform many different functions. These include, but are not limited to:

- Validating messages against a known standard.
- Generating an acknowledgement to demonstrate that a message has been received correctly.
- Extracting one or more values from a message and setting them as message properties.
- Batching or zipping multiple messages into a single file. De-batching or unzipping a file to extract its component messages.
- Encrypting or decrypting message content.
- Transforming a message to a different format. Mapping values to ensure no information is lost.

If a solution requirement cannot be met using one of Rhapsody's default filters, the JavaScript filter can be utilized to perform custom tasks with a script created as part of the solution configuration.

The Filter Toolbox

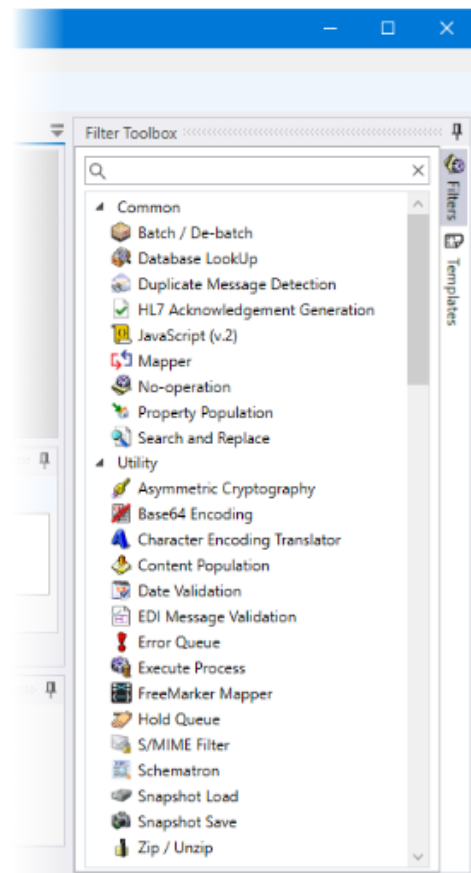
Filters are listed in the IDE's **Filter Toolbox**, displayed to the right of the route editing canvas. If the toolbox is not visible, select **Filter Toolbox** from the right-hand side of the IDE window. The **Filter Toolbox** is a pinnable element, which means it can be made always visible (pinned) or hidden when not in use (unpinned).

The **Filter Toolbox** includes more than 50 filters, which are grouped by type to simplify access. Filters can appear in more than one group.

A search box at the top of the toolbox can be used to quickly search for a filter.

The main filter groupings include:

- **Common**
These are the most frequently used filters, including No-operation, JavaScript, and Mapper filters.
- **Utility**
A variety of filters that do not fit into any other category. These include filters for encoding and validation of messages.
- **Database**
Filters for interacting with databases.
- **DICOM**
Filters to work with meta data and images in DICOM messages.
- **ebXML**
Support for messages in ebXML (e-business XML) format.
- **HL7**
Filters designed to work with HL7 version 2 and FHIR messages.
- **RGP**
Filters for working with messages that have been encrypted or signed using the PGP encryption standard.
- **X12**
Support for messages conforming with the HIPAA 5010 standard.
- **XML**
XML management tools including validation and the ability to apply an XSLT stylesheet to a message.



Other filter groups, such as **IHE** and **SAP**, are licensed separately and may not be present. New filters can be added to augment the filter tool set. In addition, site-specific filters can be developed using the Rhapsody Development Kit and included in the filter list to fulfill specific solution requirements.

Deprecated Filters

An option on the **Rhapsody Options** dialog (**Rhapsody** > **Rhapsody Options**) enables deprecated filters. These filters remain available for backwards compatibility but should not be used during the creation of new routes.

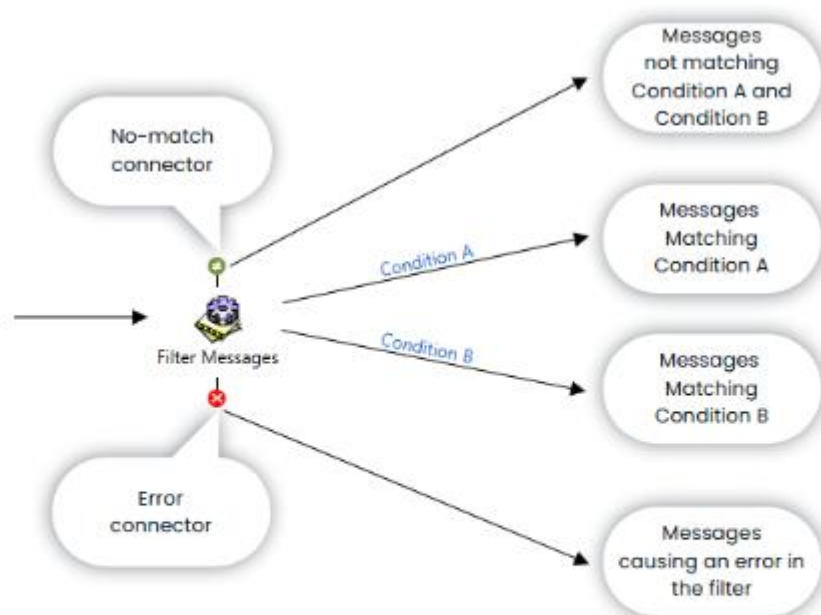
Lesson 2 of 15

Filters and Routes

Filter Attachment Points

Each filter supports four attachment points to allow connection to other components in the route:

1. **Input**
A connector entering from the left of the filter showing an inbound arrowhead.
2. **Output**
A connector leaving from the right of the filter. Conditional rules can be applied to restrict messages passing along an output connector based on their content, type or associated properties, for example.
3. **No-match connector**
The green connector above the filter. This is used to direct messages that do not match the conditions set on the filter's output connector. If no conditions are set on the output connector, the No-match connector will never be used.
4. **Error connector**
The red connector below the filter. Messages whose processing on the filter causes an error will be routed to this connector and along its attached pathway. If no pathway originates from this connector, messages routed to this connector will be sent to the Error Queue. These messages can be viewed in the Management Console.



Filters must have at least one input and one output connection. Multiple connections are possible to and from each filter. When multiple paths exit a filter, a copy of the message is sent along each path, with FIFO rules applying to all copies as a group.

Adding a Filter to a Route

A filter is added to a route by dragging its icon from the **Filter Toolbar** onto the **Route Editing Canvas**.

- If the filter is dropped on top of an existing connection, the filter will be inserted at that point in the route pathway.
- If the connector is not highlighted as the filter is dropped, the filter will simply be placed onto the canvas but will not be joined to the connector.

A filter can be dropped onto a blank area of the canvas and connections can be made to and from it by selecting the **Connector** icon and dragging the mouse pointer between components. Selected communication points and filters are highlighted by a blue rectangle. The direction of the connection is indicated by an arrow on the connector.

Filter Properties

As with communication points, filters use a tabbed **Properties** dialog with its specific configuration parameters located on the **Configuration** tab. The tabs on a filter's **Properties** dialog are:

- General
- Configuration
- Message Collector
- Notes
- Support Notes
- Auxiliary Files

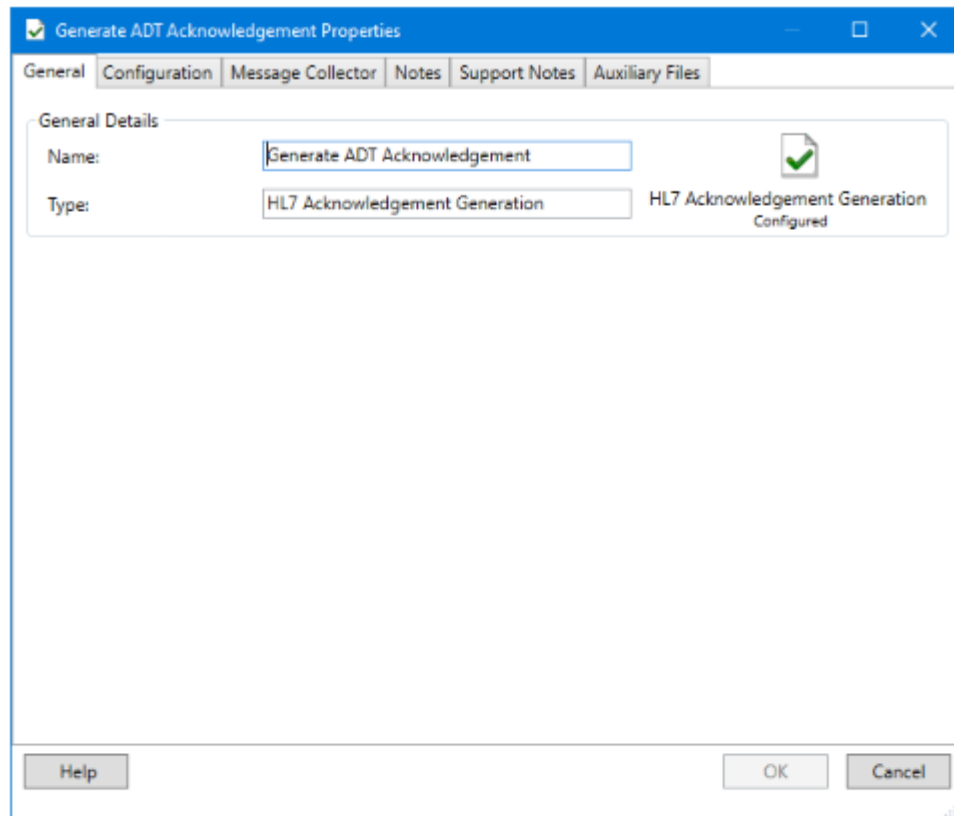
Checking in Changes

Open a filter's **Properties** dialog by double-clicking its icon in a route; the **Configuration** tab opens by default. Changing any property (on any tab) will check out the route that the filter is associated with. After the changes to the filter have been completed, check in the route to commit them to the engine.

Filters cannot be checked in independently of the route they are a part of.

General Tab

The **General** tab shows the filter's status and allows its name to be changed. The filter's **Type** is read-only and can only be changed by deleting the filter from the route and adding a different one.



 It is Best Practice to give filters and communication points names that identify their role in the solution. These names should not occupy more than four lines on the route canvas. The use of spaces rather than underscores is recommended to improve readability. The No-operation filter should always be renamed after adding it to a route.

Configuration Tab

The **Configuration** tab presents the filter's configurable properties. Contextual help for a selected property is displayed at the bottom of the dialog. The **Help** button provides a link to the documentation on the configuration and usage of the filter.

Generate ADT Acknowledgement Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

Property	Value
Message Definition *	Academy-HL7v2.4-ADT.s3d (Same as server)
Response Message Mapping	
Acknowledgement Mode *	Always ACK (AA)
NACK Mode	AR
User Description	
On Message Validation Failure *	Return Ack only to route
On Successful Validation *	Return Ack only to route
Message Errors Property Name	
HL7 Version	Same as input message
Maintain Field Length *	Ignore field lengths

Help OK Cancel

Message Collector Tab

The **Message Collector** tab provides functionality to collect messages coming from one or more sources and to process them as a batch.

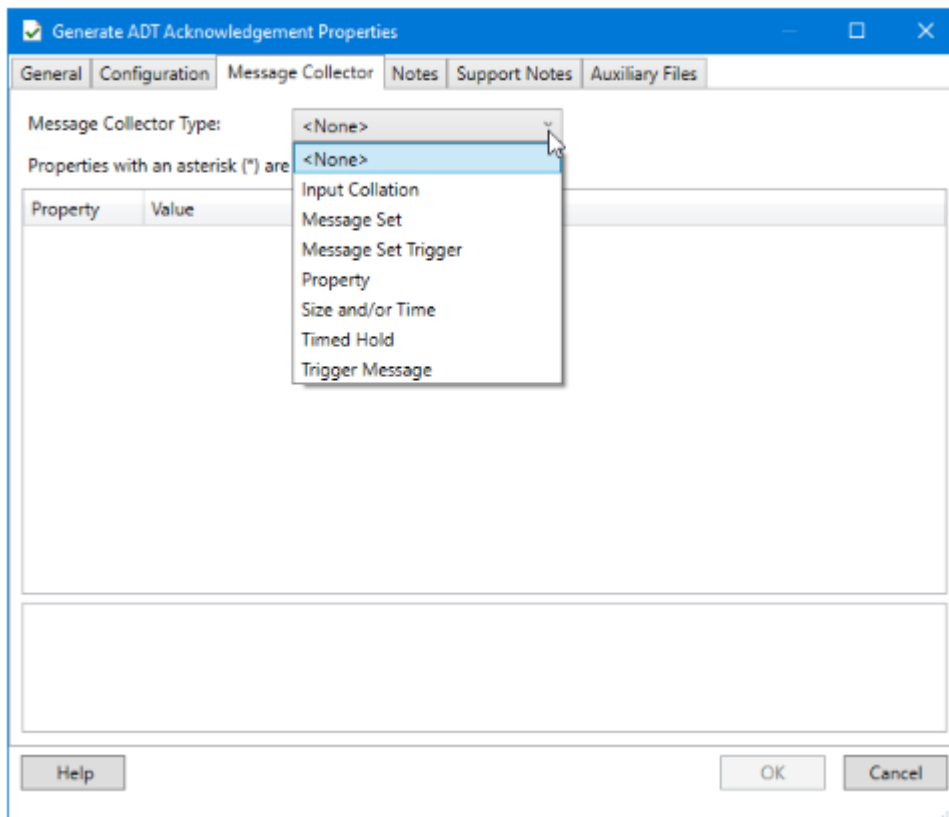
For example, when a doctor needs to see a patient's medical information, their system might send a request to Rhapsody. In the background, Rhapsody can request patient information from many different systems, each using different communication protocols such as HL7, XML or database queries. Rhapsody can then wait for all systems to respond, combine the collected patient information into a single record and send it back to the doctor's system as a single message.

Typically, the following filters are used in conjunction with a message collector:

- Batch / Debatch filter
Concatenates multiple input messages into a single batched output message.
- Zip / Unzip filter
Adds multiple input messages into a single Zip archive.
- Mapper filter
Maps multiple input messages into a single output message. This is achieved by running the main map code multiple times, once for each input message. The results from each run are stored and batched into the single output message.

- JavaScript filter

The script loops over an input collection of one or more messages. The filter can be configured to output single or multiple messages.



Message Collector Type Options

- **Input Collation**
Messages are collected with a message needing to have been received from each of the inputs to the filter.
- **Message Set**
Messages are collected until all messages in a set (as identified by a message property) have reached the filter.
- **Message Set Trigger**
Messages are collected until all messages with indexes less than the trigger message have arrived.
- **Property**
Messages are held and collated by a property, which can be set to a Rhapsody variable.

- **Size and/or Time**

Messages are collected until their combined size reaches the value identified by the **Message Set Size** property. Message collections that do not reach this size will be processed when the **Timeout Value** has been reached.

- **Timed Hold**

Individual messages are delayed from processing for a minimum guaranteed hold time.

- **Trigger Message**

Messages are collected until a message with a specific message property is received. When this occurs, all messages will be released simultaneously.

- The configuration and use of Message Collectors is described in greater detail in the *Rhapsody Professional* course.

Notes Tab

The filter's **Notes** tab provides a free text entry panel allowing documentation details to be stored with the configuration. These notes are not used by the engine but will appear in the PDF file generated when **Save Documentation** is selected from the route's context menu in the IDE **Workspace**.

In much of your configuration, the route notes should contain sufficient detail on the operation of the filters on the route. In these cases, documentation isn't required on the filter.

Where a filter contains documentation, it should include:

- **Author**

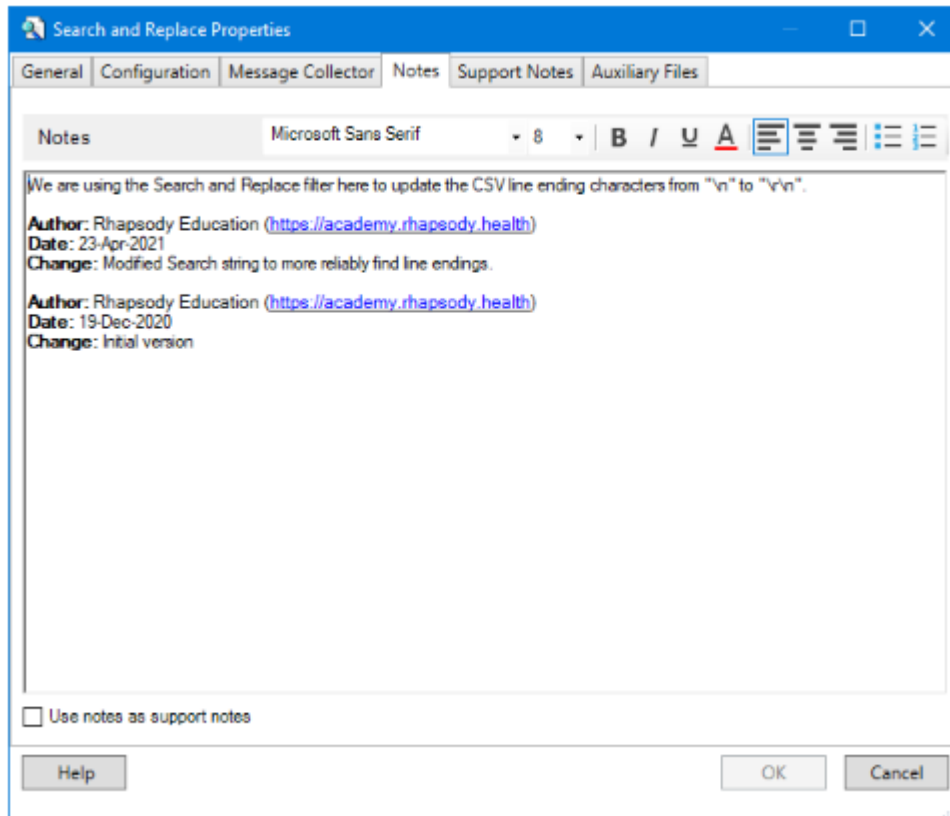
The name of the person responsible for the configuration of this filter.

- **Date**

The date when the filter was created, along with the dates of all significant amendments.

- **Purpose**

The role of this filter in the overall Rhapsody solution. This can also include any specific or unusual configuration options, or details of the associated project.

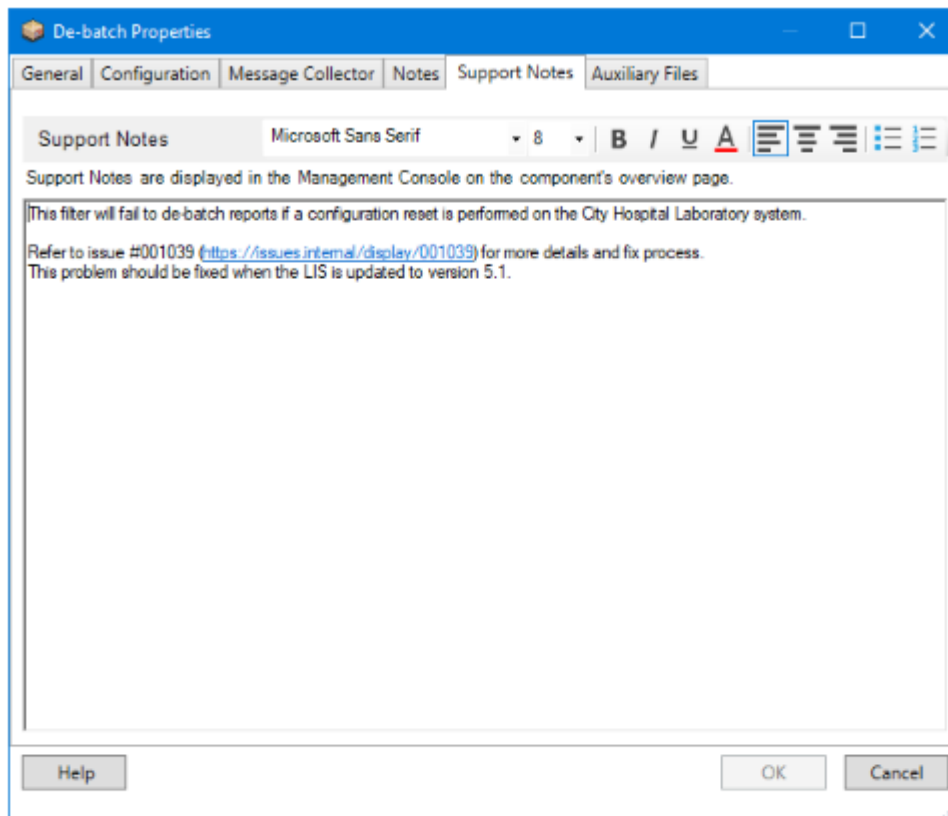


Where your notes are suitable for use as support notes, select the **Use notes as support notes** checkbox. This disables the contents of the **Support Notes** tab, and uses the contents of the **Notes** tab to populate the **Support Notes** panel on the filter's Overview screen in the Management Console.

Support Notes Tab

The **Support Notes** tab provides a free text entry panel containing details to assist when supporting a filter. These details could include:

- General information on the types of messages this filter is expected to process.
- Links to support documentation or message specifications.
- Troubleshooting tips for this filter.
- Contact details for an experienced co-worker who may be able to offer guidance on the configuration.



Viewing support notes

Information entered on a filter's **Support Notes** tab is displayed on the detail page for the filter in the Management Console.

Viewing support notes

Information entered on a filter's **Support Notes** tab is displayed on the detail page for the filter in the Management Console.

Education

7.4.0 (Development)

Administrator

10-Sep-2024 14:09

?

RHAPSODY

INTEGRATION ENGINE

Monitoring

Engine Monitor

Communication Points

Routes

Web Services

REST Clients

System Monitor

Error Queue

Hold Queue

Ports in Use

Find Messages

Input Search

Output Search

Route Search

Message Collector Search

Saved Searches

Recent Results

Notifications

Management

System State

Reports

Filter De-batch (Batch / De-batch)

Configuration

Mode	De-batch
Batch Header Expression	BHS\
Batch Trailer Expression	BTS\
Separator Type	Marks start of message
Message Separator Expression	MSH\
Index Base	Zero

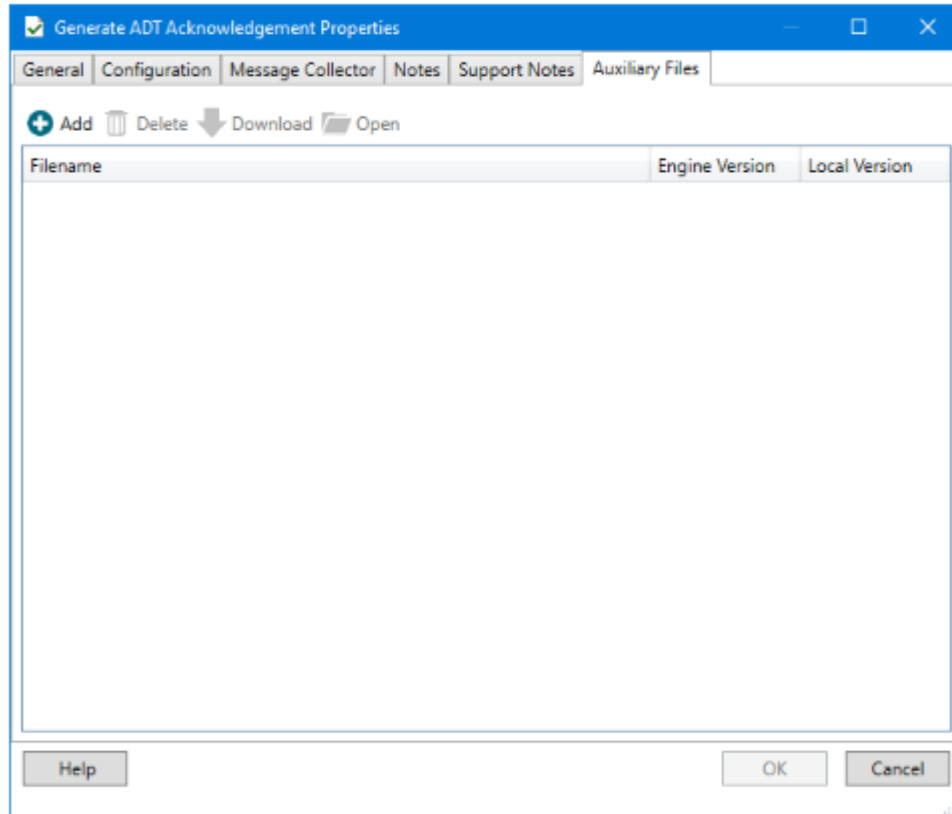
Support Notes

This filter will fail to de-batch reports if a configuration reset is performed on the City Hospital Laboratory system.

Refer to issue #001039 @<https://issues.internal/display/001039> for more details and fix process.
This problem should be fixed when the LIS is updated to version 5.1.

Auxiliary Files Tab

The **Auxiliary Files** tab manages files associated with the filter configuration.



The route containing this filter must be checked in after changes to auxiliary files have been made. This is shown when both the **Engine** and **Local** versions of the file are identical.

Lesson 4 of 15

Add a Test

Filters can be tested independently of the route they are associated with, and without starting the route or its communication points. The route containing the filter to be tested must be checked in at least once before you can test a filter (the filter configuration does not need to be checked in before testing). This feature accelerates the development and troubleshooting process for new and existing Rhapsody configurations.

The following screenshots and process description utilize the HL7 Acknowledgement Generation filter, which is designed to create and pass on an acknowledgement for every message it receives.

Filter testing is performed by selecting the **Test Filter Configuration** link on a filter's **Properties > Configuration** tab.

The screenshot shows the 'Generate ADT Acknowledgement Properties' dialog box with the 'Configuration' tab selected. The dialog has tabs for 'General', 'Configuration', 'Message Collector', 'Notes', 'Support Notes', and 'Auxiliary Files'. A note states: 'Properties with an asterisk (*) are required.' A link 'Test Filter Configuration' is visible in the top right. The main area contains a table with properties and their values.

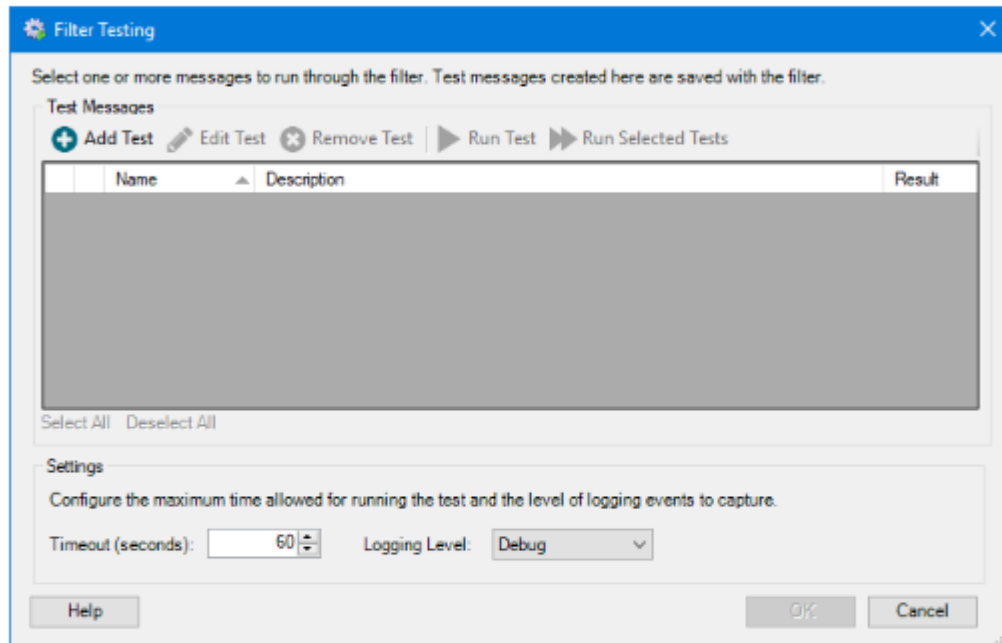
Property	Value
Message Definition *	Academy-HL7v2.4-ADT.s3d (Same as server)
Response Message Mapping	
Acknowledgement Mode *	Always ACK (AA)
NACK Mode	AR
User Description	
On Message Validation Failure *	Return Ack only to route
On Successful Validation *	Return Ack only to route
Message Errors Property Name	
HL7 Version	Same as input message
Maintain Field Length *	Ignore field lengths
Fields to Copy	[[MSH.SendingApplication,MSH.ReceivingApplication],[MSH.SendingFac

The name of the message definition file that defines the structure of all messages to be handled by this Filter.

 Filters should be tested with a range of messages, including ones that are expected to successfully complete processing and those that are expected to fail processing. Only when a full range of messages have been tested can you have confidence in the correctness of the filter's configuration.

Filter Testing

The **Filter Testing** dialog is used to add, edit and remove test messages, and to run the tests after their setup has been completed.



Test settings

- **Timeout** (seconds)
Select the maximum time allowed for running the test using the loaded message.
- **Logging Level**
Set the level of detail to include in the log as the test is run. **Trace** adds the most information, **Debug** is the default and **Fatal** includes entries for those actions which prevent the filter from working.

Filter Test Editor

Selecting **Add Test** on the previous screen displays the **Filter Test Editor**, shown below.

Filter Test Editor

General

Name: Testing with an HL7 message

Description: Uses the testMessage.edi

Input Message(s) Expected Output Message(s)

Mode: Single Message

Message Definition

Definition: [Select](#) [Clear](#)

Message Properties

	Name	Value
*		

Message Body

Mode: Show Whitespace [Load Message](#)

Encoding: Unknown / Not Set

```
MSH|^~\&|Rhapsody|Rhapsody|CDR|Test|20090917105057||ADT^A03|JCA0313|P|^  
EVN||20090917105047\r  
PID||84568-4564^^MRN^MRN~104532R^^MRN^NHR||Cardinal^John ^Q||1949111:  
PD1||Bough Family Clinic^1101|Gx10-000998^WOOD^Brandon^R^^Dr^^^^^^^QS  
PVL||I|QS-H1^^Bd003B^^^^^F1003|I|||Qx10-000424^Samson^Elizabeth^T^^Dr^^
```

Help OK Cancel

Enter the following details relating to the new or selected test message. These details represent the state of the message as it enters the filter.

Enter the following details relating to the new or selected test message. These details represent the state of the message as it enters the filter.

- **Name** (required)
A name that uniquely identifies this test.
- **Description**
Text explaining what is being tested; for example, which message type or property is being tested.

- **Definition**
The message definition that matches the structure of the test message. This can be left blank if there is a message definition associated with the filter configuration.
- **Message Properties**
Property values associated with the message when it enters the filter.
- **Load Message**
Select this link to load the test message.
- **Message Body Mode and Encoding**
Set to match the test message content and encoding. When in doubt, leave at their defaults.

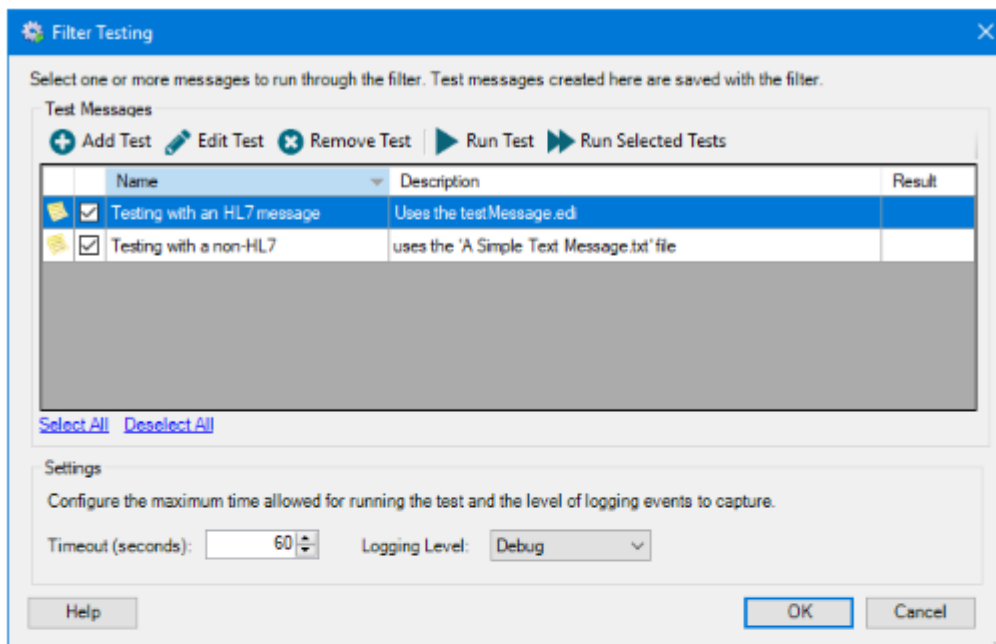
Select **OK** to load the test message onto the filter, along with the additional details entered above.

Lesson 5 of 15

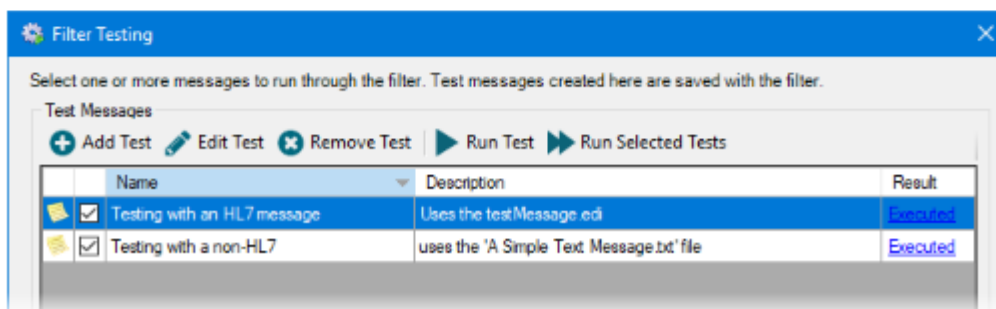
Running a Test

With the test messages added, you can run the tests against the filter.

Select a test and select **Run Test** to run just that test. Alternatively, select the checkboxes corresponding with two or more tests, then select **Run Selected Tests** to run the tests as a group. By default all tests are selected.



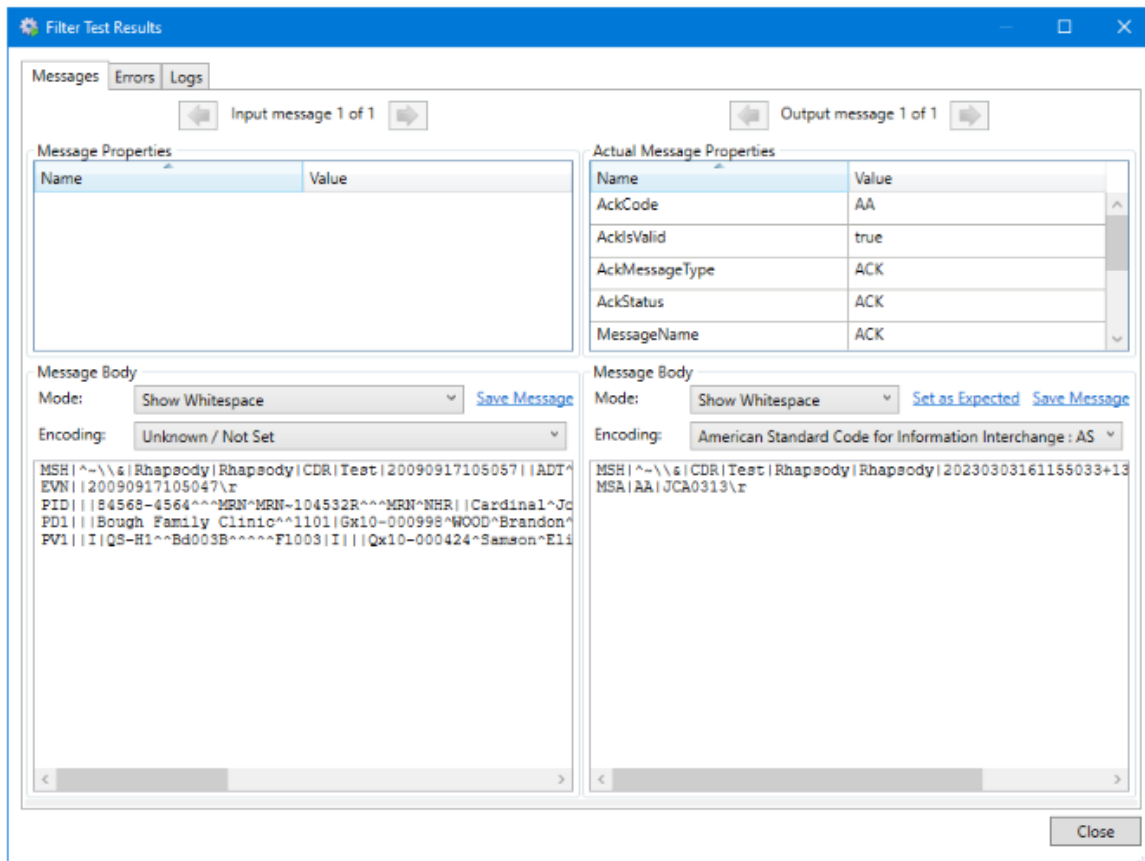
A successfully executed test will display the **Executed** link in the **Result** column.



This result tells you that processing the message by the filter was completed successfully but does not tell you whether the actual test result matched the expected outcome. For that information, you need to select the **Executed** link.

Viewing the Result

Clicking the **Executed** link opens the **Filter Test Results** window. This window shows the input message on the left, and the output message on the right. The example below suggests that no errors occurred when the message was processed by the filter.

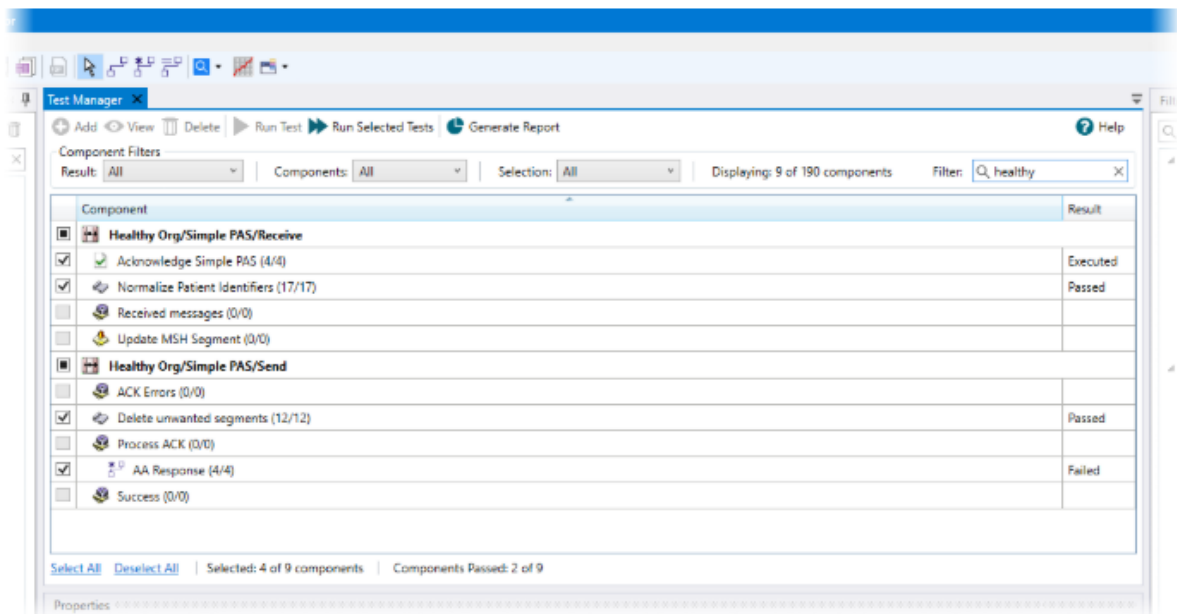


If you were to test an invalid message, the test might show a result of **Failed**. This is an expected result. An invalid test message will not have a recognized structure and therefore cannot be processed. Even though the test result indicates that a failure has occurred, the test itself was successful.

Lesson 6 of 15

The Test Manager

Rhapsody includes a **Test Manager** that allows all filters and conditional connectors with associated tests to be tested from a central location, instead of opening each filter and connector for individual testing. The **Test Manager** can be accessed from the IDE's **View** menu or by selecting the **Managers** dropdown in the IDE Toolbar.



The view can be filtered to show only those components that match specified criteria, including:

- Result**
 The overall result for all checked tests associated with the filter or conditional connector - whether Passed, Failed, Executed, Skipped or None. The overall test result is determined by the highest priority result in the group of tests. For example, for a component with three tests where one Failed and two Passed, the cumulative result is Failed.
- Components**
 Include all components in the configuration or only those with one or more associated tests.
- Selection**
 Include all components with associated tests or only those with one or more checked tests.

The count next to the components identifies the number of their associated tests (the second number) along with the number of those tests that are checked (the first number). The No-operation filter generally will not have an associated test because it makes no change to messages. There are a number of filters that do not support testing, including the Duplicate Message Detection, Hold Queue, Database Look Up and Database Message Extraction filters.

Select the **Component** title to sort the filters and conditional connectors, grouped by route. Select a second time to reverse the sort.

Test Management

The tests associated with a selected filter or conditional connector can be managed by selecting the **Add**, **Edit** or **Delete** icons at the top of the **Test Manager**.

Run tests

Select a filter or communication point in the **Component** list followed by the **Run Test** button to run just that test. Alternatively, select the component checkboxes followed by the **Run Selected Tests** button to run multiple tests simultaneously. Selecting or deselecting the checkbox next to the route's title selects, or clears the selection of, all associated tests.

Selecting a component row opens the **Filter** or **Conditional Connector Testing** dialogs where its associated tests can be selected, edited and run.

Generate report

Selecting **Generate Report** generates a document in PDF format containing a summary of the components with and without associated tests, along with the results of running those tests. The following screenshot shows a summary of the tests configured on the previous pages of this module.

Testing Summary

Summary	
Total number of components	190
Total number of components with tests.	130
Total number of Failed components	1
Total number of Passed components	74
Total number of components with no expected output executed successfully	7

Healthy Org/Simple PAS

 Send

Filter: Process ACK	
Connector: AA Response (4/4)	Failed

An example Test Manager Report

The only option when generating the report is whether to include a **Title Page**. The report's footer includes the date and time on when the report was run, plus the Rhapsody version and the name of the Rhapsody server.

Lesson 7 of 15

Introduction

In this section we'll look at the following commonly used filters. We'll investigate their configuration, then review them via filter testing and the Management Console.

No-operation

[Go to Exercise](#)

HL7 Acknowledgement Generation

[Go to Exercise](#)

Content Population

[Go to Exercise](#)

Property Population

[Go to Exercise](#)

EDI Message Validation

[Go to Exercise](#)

Hold Queue

[Go to Exercise](#)

JavaScript

[Go to Exercise](#)

Resources

These exercises use the messages and message definitions available in the `.\Rhapsody\Associate Files and Definitions\106 - Filter` folder in the resources Zip file available at the start of the course.

Exercise - No-operation

This exercise introduces the No-operation filter. There are no steps you need to follow; it is just informational.

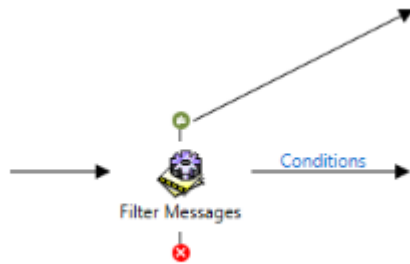
The No-operation filter has no effect on messages passing through it, but is very important in many solutions. It is commonly used to control a message's pathway through a route or to clarify and document the route's visual layout.

The No-operation filter has no configurable parameters.

Path Management

Path management is controlled by using conditional connectors on the No-operation filter's output pathway(s). These allow routing decisions to be made based on the message's content or associated properties. When a conditional connector is applied, only messages matching the conditions can pass through the connector.

- If there are no other available output connectors, non-matching messages will be sent to the **No-match** connector.
- If there is no connection attached to the **No-match** connector, the message will be sent to the **Error** connector.
- If there is no connection on the **Error** connector, the message will be sent to the Error Queue.

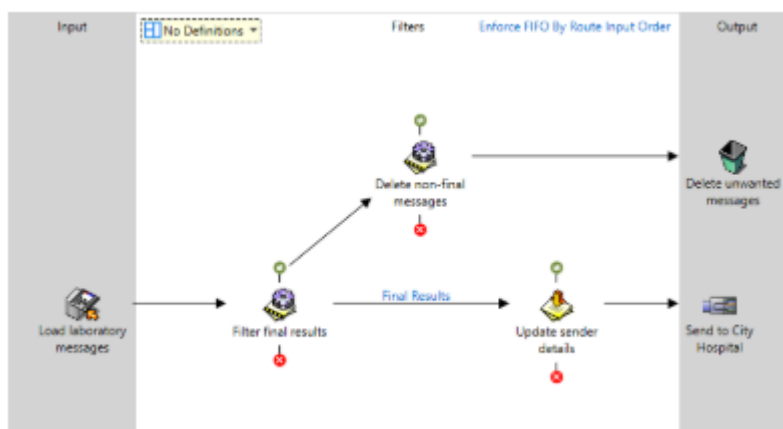


Route Labelling

The No-operation filter is often added to a route to provide a label to assist the quick interpretation of message flow. In this example, the No-operation filter has been added in two places. The first use of the filter lets us know that results are being filtered, while the second use helps us quickly identify that non-final messages will be deleted.

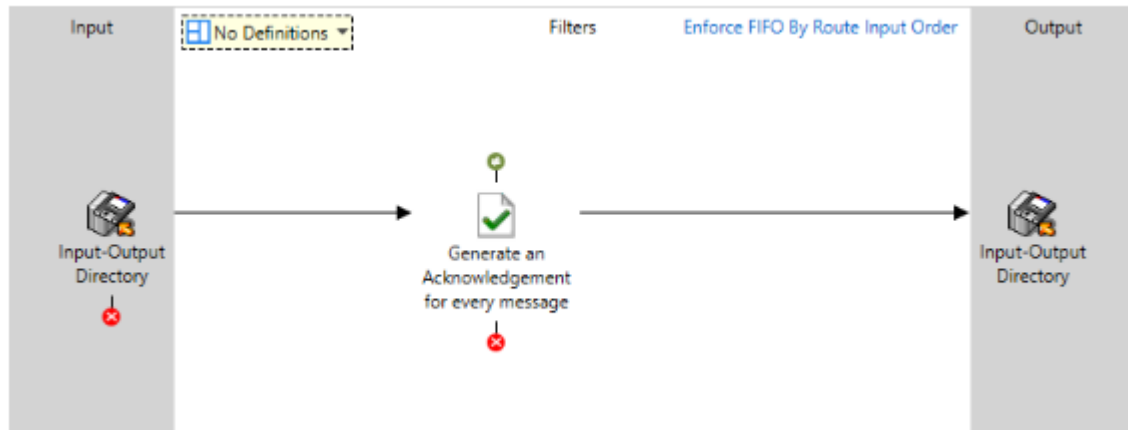
In a previous module, we looked at the effect of adding the No-operation filter to a route that has an associated message definition. The addition of this filter forces messages to be parsed by the engine as they move through the route, resulting in message properties being implicitly added to the message.

This will occur whenever any filter is added to a route, not just the No-operation filter.



Exercise - HL7 Acknowledgement Generation

The HL7 Acknowledgement Generation filter provides the ability to validate an incoming HL7 message against a specified definition and to generate a matching acknowledgement that can be returned to the sending system.



Configure the Route

1. 1

Create a folder for the components of this exercise named *HL7 Acknowledgement Generation*.

2. 2

Create a route with a descriptive name that matches, for example, its containing folder.

3. 3

Create a directory communication point named *Input-Output Directory* and configure it as follows:

- **Mode**
Bidirectional
- **Input Directory**
\$(RhapsodyAssociate_TestDir)\REN-106\ACK_Generation\Input
(or similar)
- **Output Directory**
\$(RhapsodyAssociate_TestDir)\REN-106\ACK_Generation\Output
(or similar)

Other properties can be left at their default values.

4. 4

Add the newly created communication point to the input and output of your route.

Configure the Filter

1. 1

Open the HL7 Acknowledgement Generation filter's **Properties** dialog. On the **Configuration** tab and specify the following property values:

The screenshot shows a Windows-style dialog box titled "Generate an Acknowledgement for every message Properties". It has several tabs: "General", "Configuration" (which is selected), "Message Collector", "Notes", "Support Notes", and "Auxiliary Files". Below the tabs, there is a note: "Properties with an asterisk (*) are required." and a link "Test Filter Configuration". The main area contains a table with two columns: "Property" and "Value".

Property	Value
Message Definition *	Academy-HL7v2.4-ADT.s3d (Server version modified)
Response Message Mapping	
Acknowledgement Mode *	Always ACK (AA)
NACK Mode	AR
User Description	
On Message Validation Failure *	Return Ack only to route
On Successful Validation *	Return Ack only to route
Message Errors Property Name	
HL7 Version	Same as input message
Maintain Field Length *	Ignore field lengths
Fields to Copy	[[MSH.SendingApplication,MSH.ReceivingApplication],[MSH.SendingFacility
Use Custom DateTime Format	☐
Custom DateTime Format *	

At the bottom of the dialog, there are three buttons: "Help", "OK", and "Cancel".

- **Message Definition**

Select the ellipsis '...', then select *Academy-HL7v2.4-ADT.s3d*. This definition should already be loaded into the engine. If not, a copy can be found in the exercise resources.

The other properties can be left at their default values. The descriptions below explain their function.

- **Response Message Mapping**

The matching acknowledgement for each input message type. When left blank, a generic ACK will be returned for every incoming message - regardless of message type.

If, for example, the incoming message is an HL7 ADTA03 (Discharge patient message), its matching acknowledgement will be an ACKA03, as shown below. An 'Unknown Message Found' error will result if the matching acknowledgement type is not present in the loaded message definition.

Input Message	ACK Message
ADTA03	ACKA03

- **Acknowledgement Mode**

The type of acknowledgement to be generated for each incoming message. The default is **Always ACK (AA) - Acknowledgement Accepted**, which means that the original message is assumed to be error-free.

- **User Description**

A text message that will be inserted into the MSA/TextMessage field of the generated acknowledgement. The default is to populate nothing into this field.

- **On message validation failure**

The action to take when an incoming message fails validation against the supplied message definition. The options are:

- Return Ack only to route (default)
- Return input message only to route
- Send input message to error connector
- Return both input message & Ack to route

- **On successful validation**

The action to take when an incoming message is successfully validated. The same options as the **On message validation failure** property are available.

- **Fields to Copy**

The fields in incoming messages to be copied to their corresponding locations in

the generated acknowledgement. The default matches the standard requirements for an HL7 Acknowledgement message.

1. 2

Check in all changes.

Test the Filter

Now we'll test the filter's configuration. This does not require the route to be started.

1. 1

On the filter's **Configuration** tab, select the **Test Filter Configuration** link.

2. 2

On the **Filter Testing** dialog, select **Add Test**.

3. 3

On the **Filter Test Editor** dialog, enter a name for the test message along with an optional description.

4. 4

Select the **Load Message** link to load the *testMessage.edi* test message, then select **OK**.

5. 5

On the **Filter Testing** dialog, select **Run Selected Tests**.

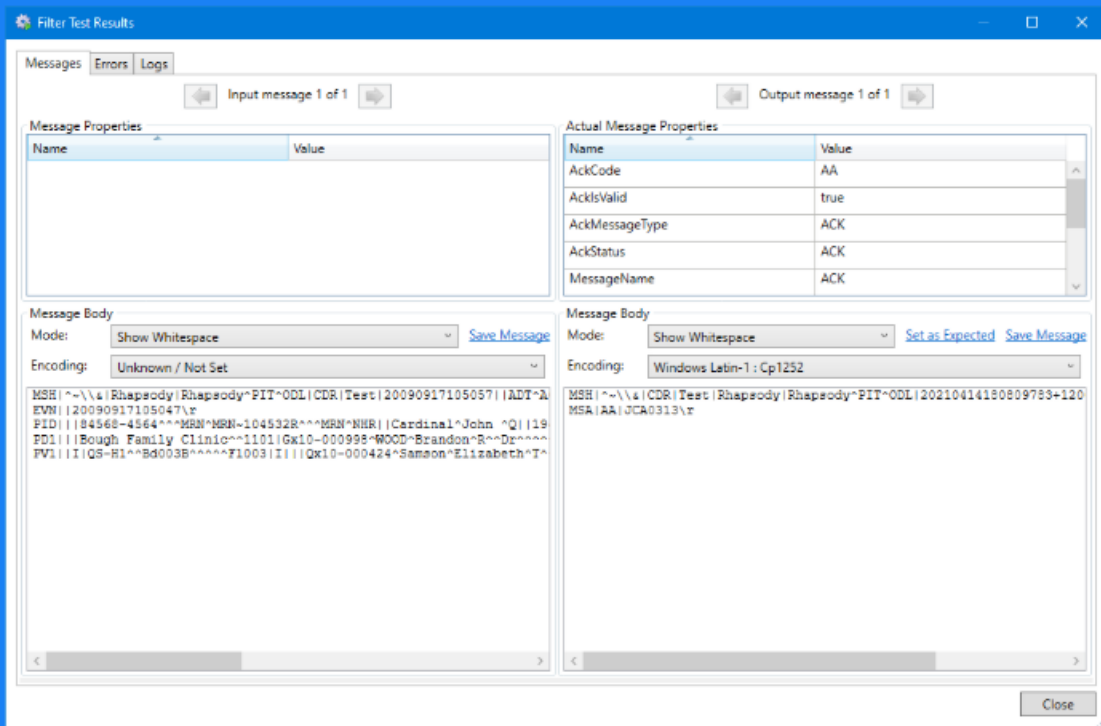
6. 6

View the **Input** and **Output** messages on the **Filter Test Results** dialog.

The result should be similar to that described in Filter Testing earlier in this module.

Model Answer

A successful test of the filter shows the incoming **ADTA03** message in the panel on the left of the screen, while the acknowledgement generated from it is shown on the right. The **AckCode** property value **AA** matches the **Acknowledgement Mode** set on the filter's **Configuration** tab.



Test the Route

- 1

Start the route and communication point.

- 2

Place a copy of the test message into the communication point's input directory.

- 3

Check that after a brief period an acknowledgement is generated and copied to its output directory.

View the Result

- 1

From the **Monitoring** menu, select **Communication Points**.

2. 2

Select the Directory communication point in the Acknowledgement Generation route you used for this exercise.

3. 3

Open the communication point's **Output Archive**, then select the most recently processed message.

4. 4

On that message's **Message View** screen, select the HL7 Acknowledgement Generation filter's **Event Actions** icon, followed by **Compare In and Out**.

You should be able to view the structure and content of the message as it arrives and leaves the HL7 Acknowledgement Generation filter. The results should match what you saw above when you tested the filter independently of the route.

Split Screen View

To display the split screen view shown below, select the **Event Action** icon in the right-hand column of the HL7 Acknowledgement Generation filter's entry in the **Message Events** tree. Select **Compare In and Out** to simultaneously view the input and output messages for this filter.

Message View 1.26658865395712.51

Displaying message 1 of 8 from the result set named 'Input-Output Directory Output Archive (2)' [Back to Result Set](#)

[Reinject](#) [Redirect Copy](#) [Redirect Edited Copy](#)

Message Events

[Always Show Outputs](#) Last refreshed at 12:23

COMPONENT	TIMESTAMP	DESCRIPTION
 Input-Output Directory	Apr 19, 2021 12:23:21.025	Received by communication point
 Test Route	Apr 19, 2021 12:23:21.028	Placed on route
 Generate an Acknowledgement for every message	Apr 19, 2021 12:23:21.033	Filtering succeeded, took 2.509 ms
 Input-Output Directory	Apr 19, 2021 12:23:21.033	Delivered to communication point
	Apr 19, 2021 12:23:21.037	Sent by communication point, total p...

Navigation: < << >> >

Message Properties

In

Property Name	Property Value
BaseFilename	testMessage
directory.BodySize	409
FileCreationTime	1618791799693
FullFilename	testMessage.edi
InputTime	1618791801025
rhapsody.InputCommunicationPo...	Input-Output Directory
rhapsody.InputCommunicationPo...	7643
Suffix	.edi

Showing 8 of 8

Out

Property Name	Property Value
AckCode	AA
AckIsValid	true
AckMessageType	ACK
AckStatus	ACK
BaseFilename	testMessage
directory.BodySize	409
FileCreationTime	1618791799693
FullFilename	testMessage.edi
InputTime	1618791801025
MessageName	ACKA03

[Show More](#)

Showing 10 of 16

Message Body

Tree View

Flat Wire

In

Show Whitespace ☐ Encoding: windows-1252 Size: 409 bytes

```
MSH|^~\&|Rhapsody|Rhapsody|CDR|Test|20090917105057||ADT^A03|JC,EW||20090917105047|n|PID||84568-4564^^^PRN^PRN-104532R^^^PRN^MR||Cardinal^John^Q|P01||Bough Family Clinic^^1101|6x10-000908^WOOD^Brandon^R^^Dr^PW1||I|QS-H1^^000038^*****F1003|I||Qx10-000424^Sanson^Elizabeth
```

Download

Out

Show Whitespace ☐ Encoding: windows-1252 Size: 94 bytes

```
MSH|^~\&|CDR|Test|Rhapsody|Rhapsody|20210419122521055+1200||AC|MSA|AA|JCA0313|n
```

Download

Exercise - Content Population

The Content Population filter is used to insert information into a message. There are two options for how this information can be inserted:

1. Field Mappings

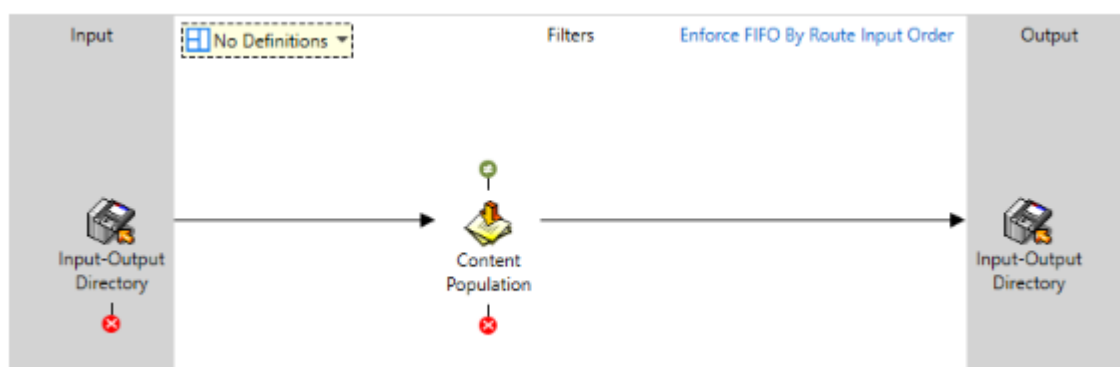
The content of individual fields within the message is replaced by the specified information. Use of this option requires the use of a message definition matching the structure of incoming messages to have been loaded onto the filter.

2. Body Template File

The content of the incoming message is entirely replaced by the content of the specified file.

Replace Content using Field Mappings

This exercise uses the Content Population filter to replace a field in an incoming HL7 ADT test message.



Configure the route

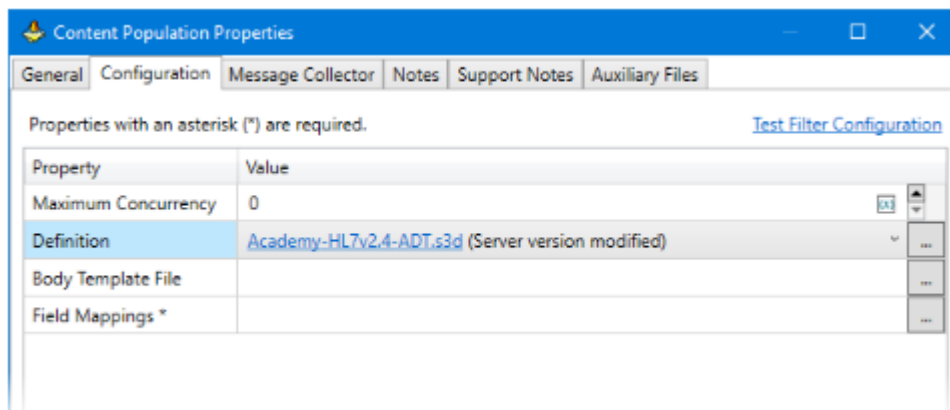
Create the above route by repeating a modified version of the steps you used to create the **HL7 ACK Generation** route earlier in this module.

1. 1

Open the **Configuration** tab of the Content Population filter's **Properties** dialog.

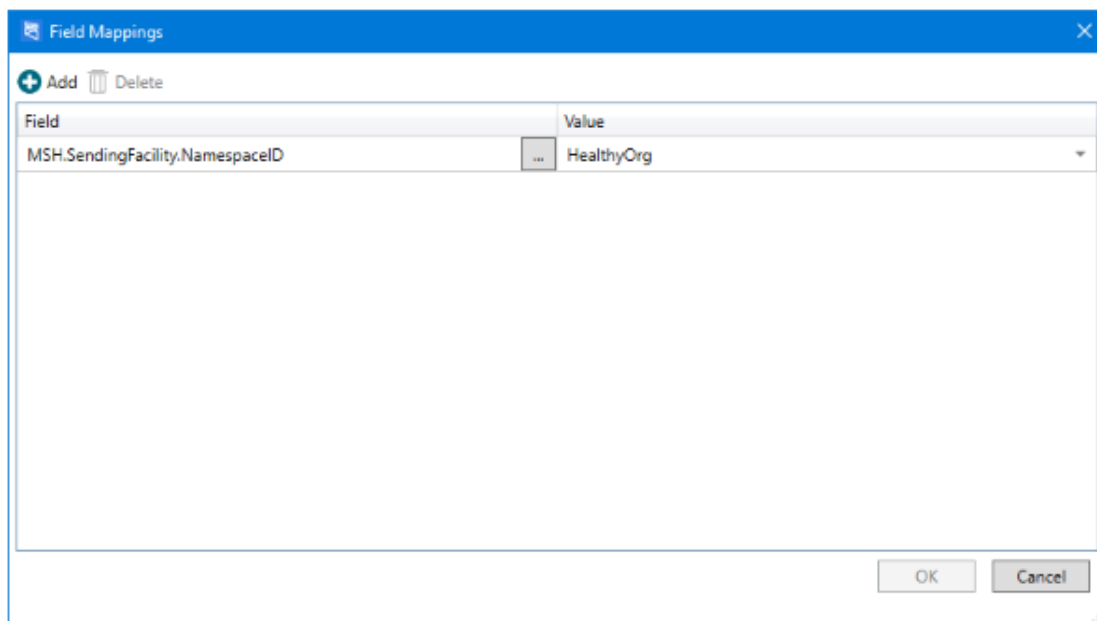
2. 2

Select the **Browse** button to the right of the **Definition** field to locate the *Academy-HL7v2.4-ADT.s3d* message definition, which should already be in the engine.



3

Select the **Browse** button corresponding with the **Field Mappings** property and, on the resulting dialog, identify the field to be populated and the value it should have.



4

Select **OK** to close the screen, then select **Apply** to apply the configuration change. There is no limit to the number of fields that can be populated using this filter.

Test the filter

- 1 From the filter's **Properties** dialog select the **Configuration** tab.
- 2 Select the **Test Filter Configuration** link, then select **Add Test**.
- 3 Provide a **Name** and optional **Description** for the test message.
- 4 Select the **Load Message** link to load the *testMessage.edi* file.

The screenshot shows the 'Filter Test Editor' dialog box with the following sections:

- General**:
 - Name: Valid ADT^A03
 - Description: REN-106 - Sample ADT^A03 file
- Input Message(s)** / **Expected Output Message(s)**:
 - Mode: Single Message
 - Message Definition: Definition: [empty] [Select](#) [Clear](#)
 - Message Properties: A table with columns 'Name' and 'Value'. The first row has a '*' in the 'Name' column. The rest of the table is greyed out.
- Message Body**:
 - Mode: Show Whitespace
 - Encoding: Unknown / Not Set
 - [Load Message](#)
 - Message content: MSH|^~\&|Rhapsody|Rhapsody|CDR|Test|20090917105057||ADT^A03|JCA0313|P|: ^ EVN||20090917105047\r PID||184568-4564^^MRN^MRN~104532R^^MRN^NHR||Cardinal^John ^Q||1949111: PD1||Bough Family Clinic^^1101|Gx10-000998^WOOD^Brandon^R^^Dr^^^^^^^QS^ < >

Buttons at the bottom: Help, OK, Cancel.

Adding a new test message for filter testing

5

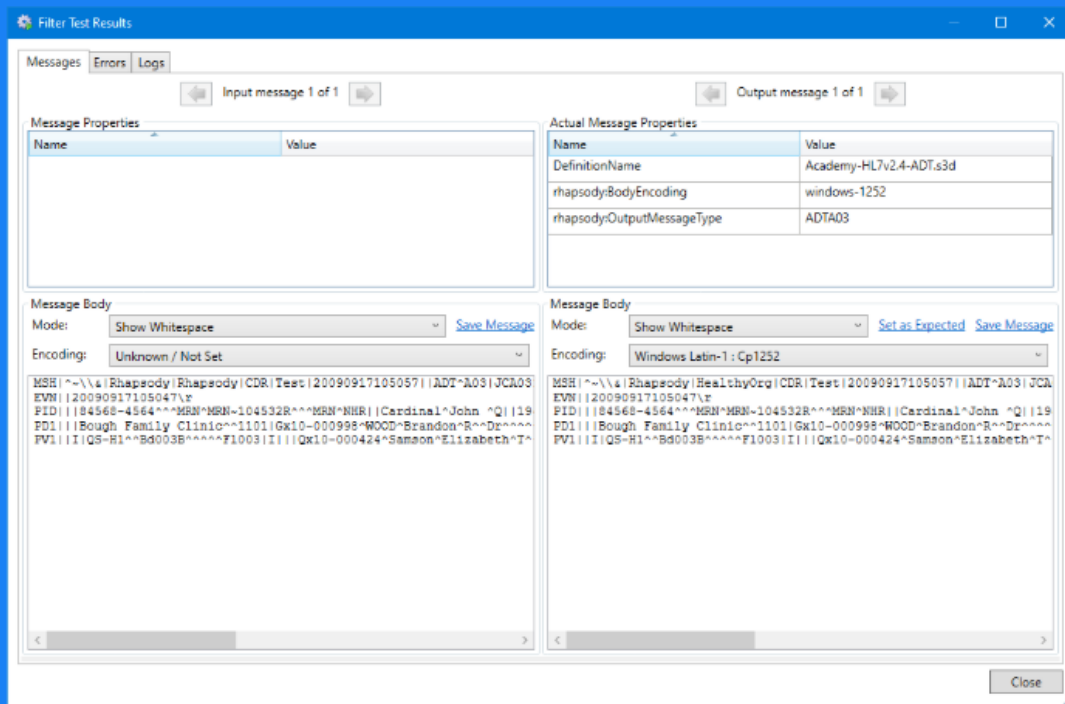
Select **OK** to save the test and run the test.

6

Select **Run Selected Tests** to test the filter using the test you have just added.

Model Answer

The expected result after updating specific fields with content specified in the field mappings is shown below. The value of the `SendingFacility.Namespace` field in incoming messages is replaced with **HealthyOrg** in outgoing messages.



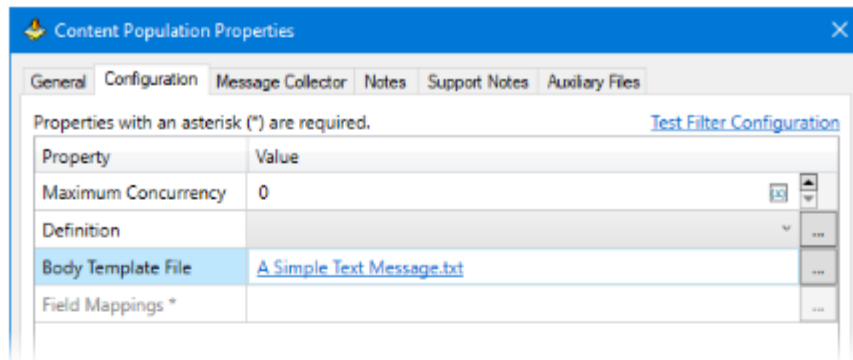
Replace Content from a Body Template File

This exercise uses the Content Population filter to replace the contents of an incoming HL7 ADT test message with the contents of a simple text message.

Configure the filter

Open the Content Population filter's **Properties** dialog and view the **Configuration** tab. Delete any existing configuration from the previous section of this exercise by removing the **Definition**.

Specify the following values:



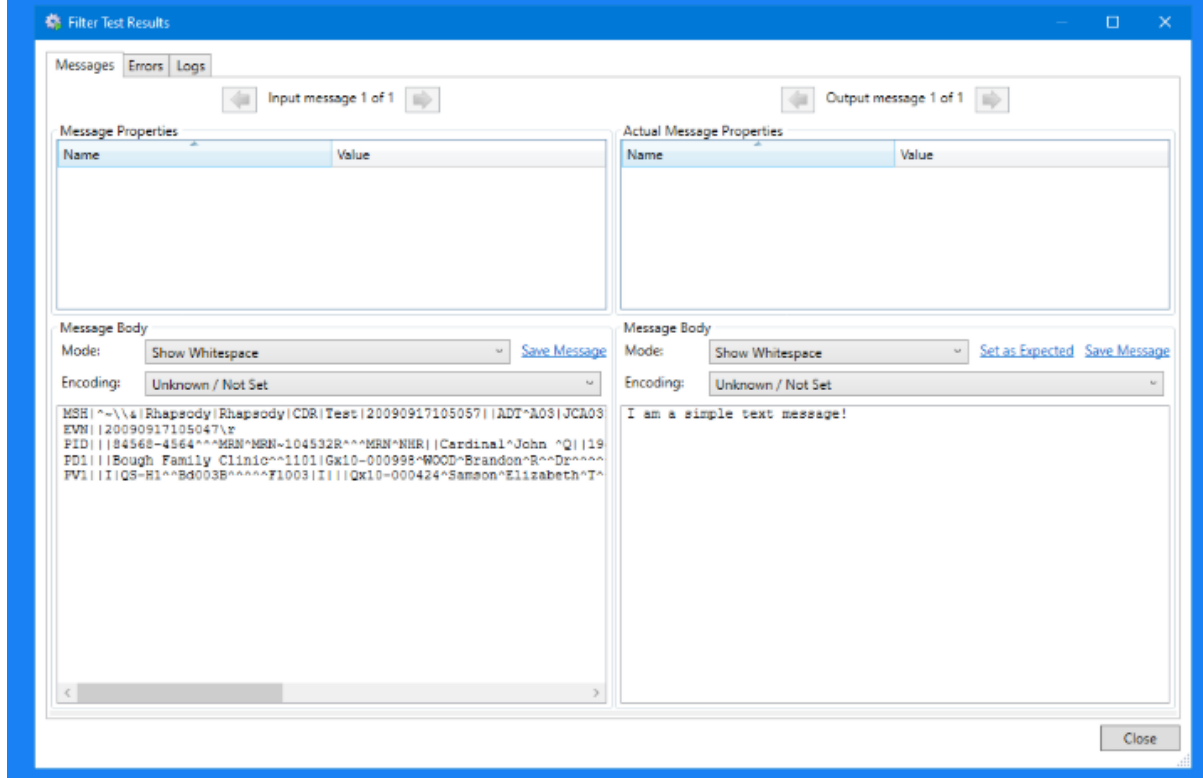
- 1 Select **Browse** to the right of the **Body Template File** field to locate and select the *A Simple Text Message.txt* file in the exercise resources, or you can create one from scratch.
- 2 Select **OK** to complete the configuration.
- 3 Check in the change. When the check-in is complete, the message *Same as server* will be displayed next to the filter to confirm that the local and server versions of the **Body Template File** are identical.

Test the filter

- 1 From the filter's **Properties** dialog select the **Configuration** tab.
- 2 Select the **Test Filter Configuration** link.
- 3 Make sure the *testMessage.edi* file has been added as a test message.
- 4 Select **Run Selected Tests** to test the filter.

Model Answer

The expected result after replacing the incoming test message with the contents of the **Body Template File** is shown in the following split screen view.



Lesson 11 of 15

Exercise - Property Population

The Property Population filter is used to insert information into one or more properties (metadata) associated with a message. There are two ways this information can be sourced:

1. From the contents of a specified field within the message.
2. Hard coded as part of the filter's configuration.

Populating from a Message Field



Configure the route

Create the above route by repeating a modified version of the steps you used to create the **HL7 ACK Generation** route earlier in this module.

Configure the filter

Open the Property Population filter's **Properties** dialog and, on its **Configuration** tab, specify the following values:

1. 1

Select **Browse** to locate and select the *Academy-HL7v2.4-ADT.s3d* message definition. This is needed by the filter to understand the structure of the incoming messages.

2. 2

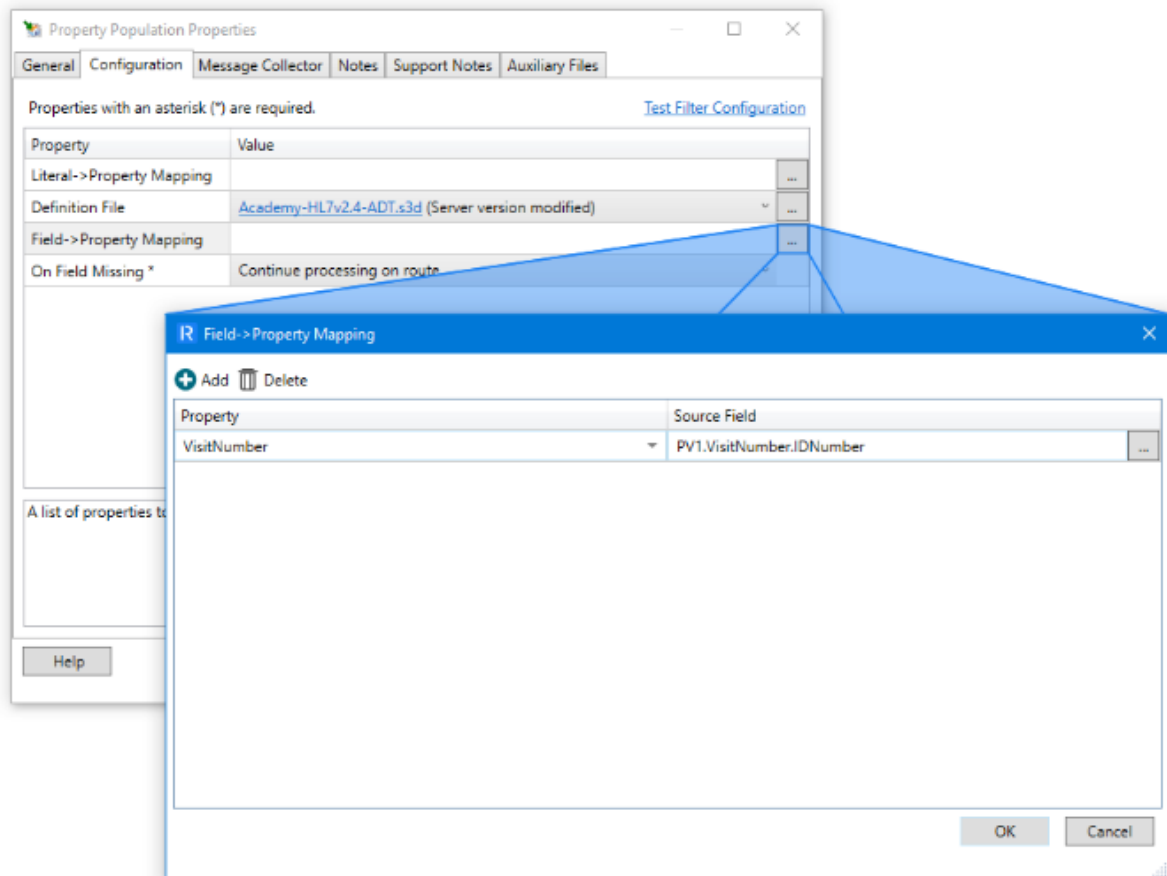
Select **Browse** to the right of the **Field -> Property Mapping** field, then on the resulting dialog, enter the name of the new message property to create.

3. 3

Select **Browse** next to the **Source Field**. Locate the ADTA03 message type and select the PV1.VisitNumber.IDNumber field. This exercise uses the visit number as the value of the VisitNumber property that will be added to incoming messages.

4. 4

Select **OK** twice to close the dialogs, then check in all changes.



Configuring the property mapping

Test the filter

1. 1

Open the filter's **Properties** dialog and select the **Configuration** tab.

2. 2

Select the **Test Filter Configuration** link, then select **Add Test**.

3. 3

Provide a **Name** and optional **Description** for the test message.

4. 4

Select the **Load Message** link to add the *testMessage.edi* file.

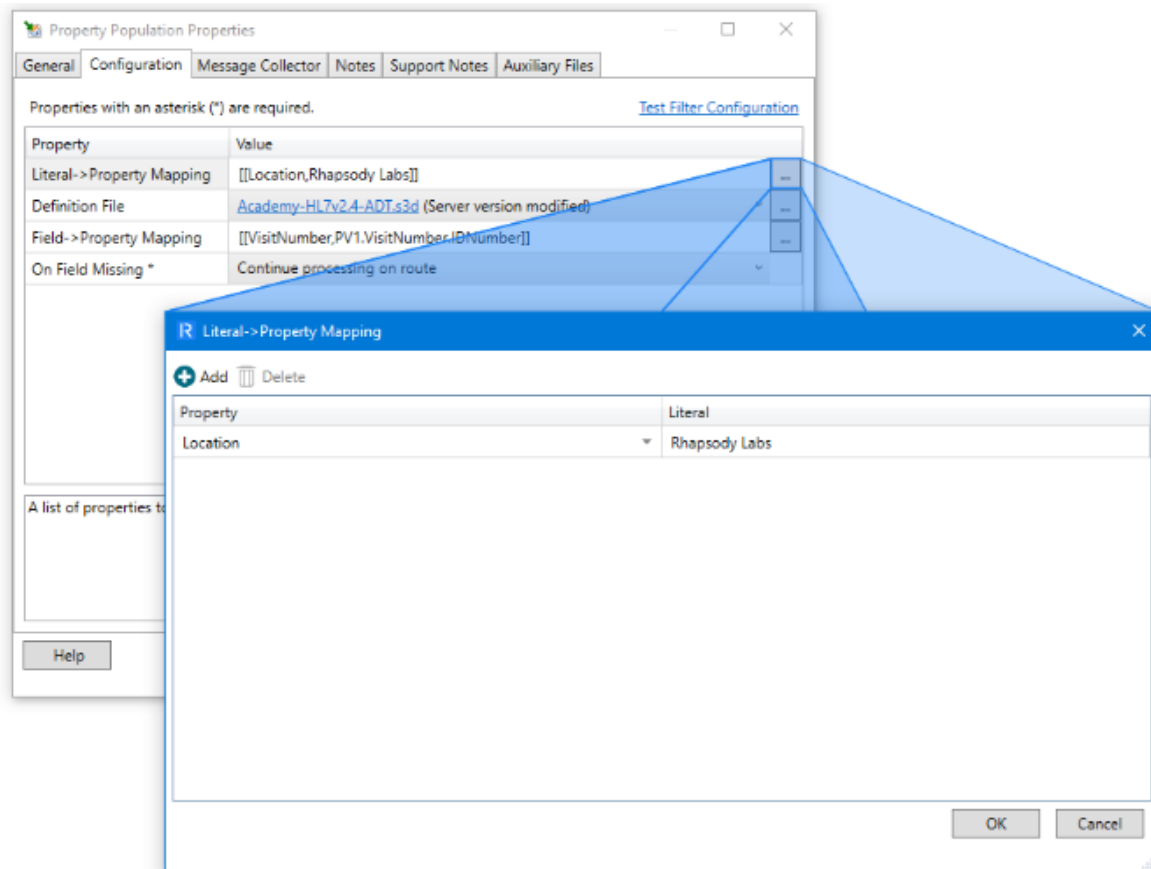
5. 5

Select **OK** and run the test.

View the Model Answer at the end of Populating a Hard Coded Value to see a screenshot of the expected result.

Populating a Hard-Coded Value

Modify the Property Population filter's configuration to match the following screenshot:



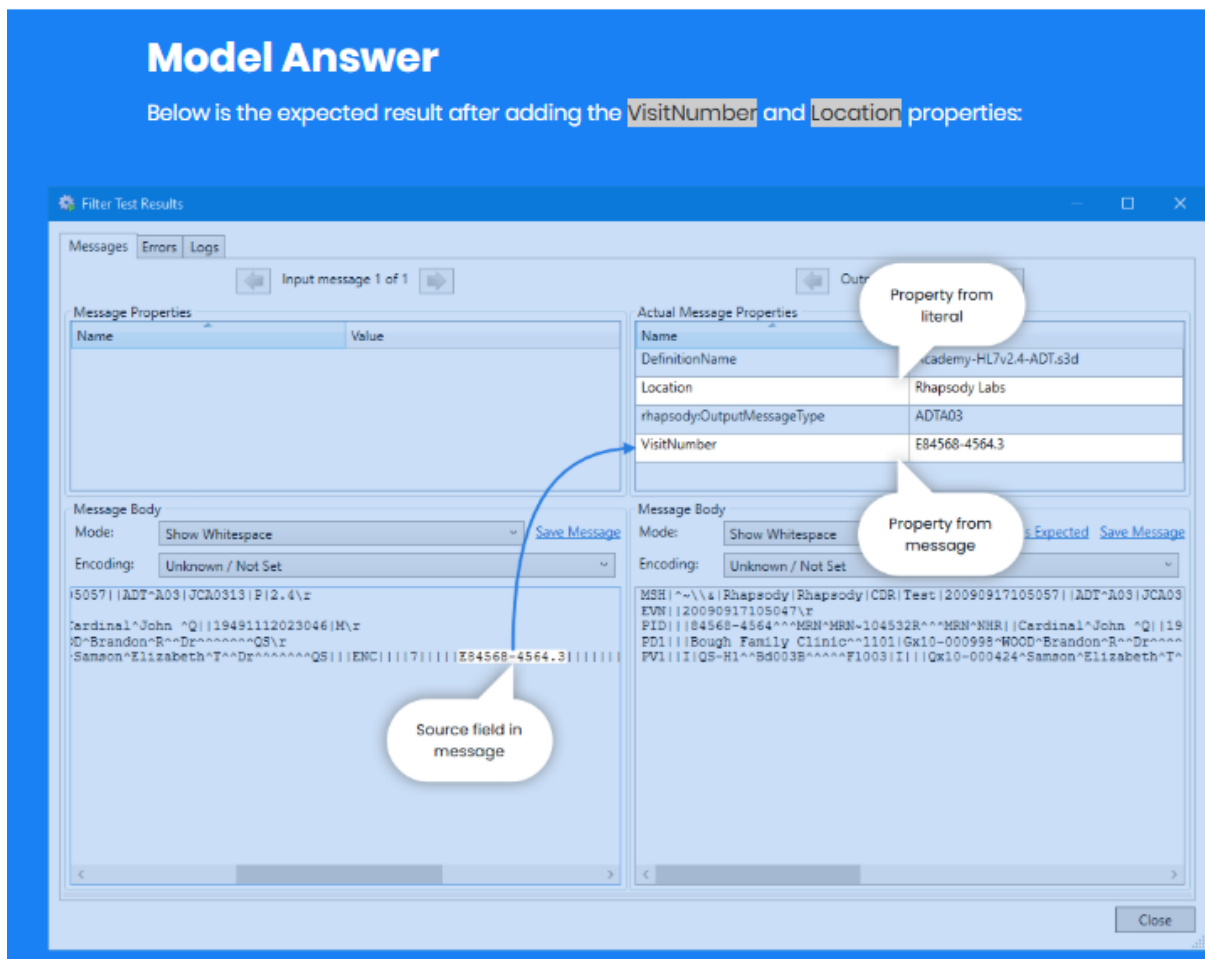
Configuring the literal mapping

Test the filter

- 1 Open the filter's **Properties** dialog and select the **Configuration** tab.
- 2 Select the **Test Filter Configuration** link, then select the **Add Test** button.
- 3 Provide a **Name** and optional **Description** for the test message.
- 4 Select the **Load Message** link to add the *testMessage.edi* file.
- 5 Select **OK** and run the test.

Model Answer

Below is the expected result after adding the **VisitNumber** and **Location** properties:



Using a Message Property as an Output Filename

We will now use the contents of the \$VisitNumber property to set the output filename for messages processed by this route.

- 1

Open the **Properties** dialog, **Configuration** tab for the Input-Output Directory bidirectional communication point used in this route.

2. 2

Configure the following properties:

- **Base Filename**
\$VisitNumber.
- **Suffix**
edi. Required to view the output file in a text editor such as Notepad++.

3. 3

Check in the changes and ensure all components have started correctly.

4. 4

Place a copy of the *testMessage.edi* file into the route's Input directory.

Model Answer

The expected result after extracting the `PVI.VisitNumber.IDNumber` field from an input file and its use as the output file's **Base Filename** is a file in the output directory with a name that matches the visit number.

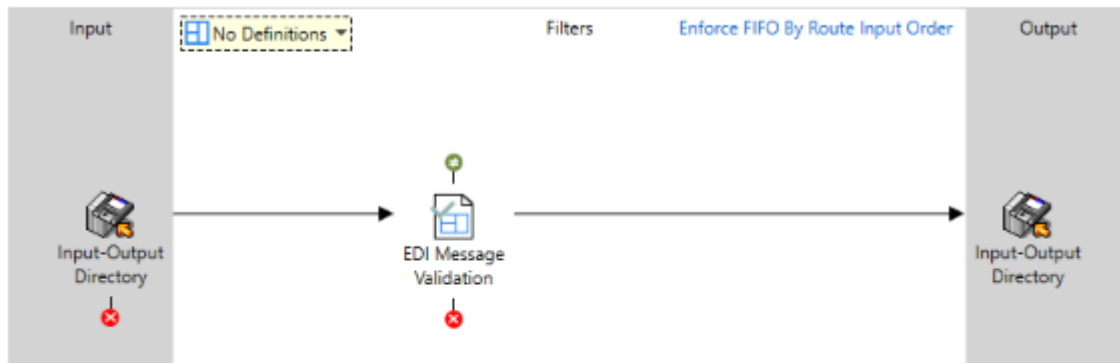
What would you expect to see if multiple copies of the test message are placed into the Input directory?



The EDI Message Validation filter is used to validate incoming EDI messages against a message definition. Processing by this filter automatically associates the ValidationResult and ValidationErrorCount properties with the message, which can be viewed in the Management Console.

Testing the Filter with an Invalid Message

Create the following route by repeating a modified version of the steps you used to create the **HL7 ACK Generation** route earlier in this module.



Configure the filter

Open the EDI Message Validation filter's **Properties** dialog. On the **Configuration** tab, specify the following values:

The screenshot shows the 'EDI Message Validation Properties' dialog box with the 'Configuration' tab selected. The dialog has tabs for 'General', 'Configuration', 'Message Collector', 'Notes', 'Support Notes', and 'Auxiliary Files'. Below the tabs, there is a text area with the message 'Properties with an asterisk (*) are required.' and a link 'Test Filter Configuration'. A table lists properties and their values:

Property	Value
Maximum Concurrency	0
Definition File *	Academy-HL7v2.4-ADT.s3d (Server version modified)
On Validation Failure *	Send to error connector
On Parse Failure *	

Below the table are two empty text areas. At the bottom of the dialog are buttons for 'Help', 'OK', and 'Cancel'.

1. Select **Browse** to the right of the **Definition** field to add the *Academy-HL7v2.4-ADT.s3d* message definition.
2. 2

View the options relating to the **On Validation Failure** property, but do not change its value from the default:

- **Send to Error Connector**
If this connector does not have an ongoing path, invalid messages will be sent to the **Error Queue** from where they can be viewed in the Management Console.
- **Continue processing on route**
The message is passed on to the next component in the route to be processed as normal.

- **Send error report to route**

A report in XML format is generated and passed into the route. If this option is selected, two further options become available from the **On Parse Failure** drop-down list:

- **Send to Error Connector**

Send the report to the error connector, from where it is passed to the Management Console's **Error Queue** or to a connected filter or communication point.

- **Continue processing on route**

Send the error report to the next component in the route.

Test the filter

1. 1

Open the filter's **Properties** dialog and select the **Configuration** tab.

2. 2

Select the **Test Filter Configuration** link, then select **Add Test**.

3. 3

Provide a **Name** and optional **Description** for the test message.

4. 4

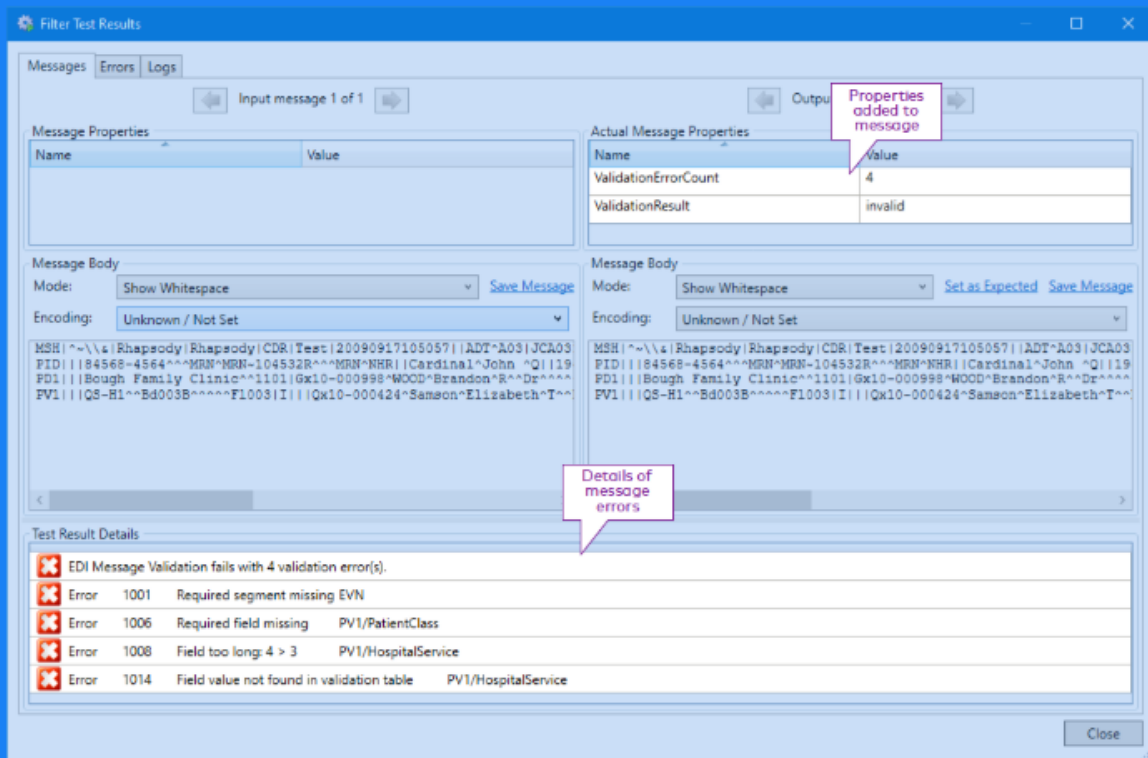
Select the **Load Message** link and add the *testMessage - Validation Error.edi* file.

5. 5

Select **OK** and run the test.

Model Answer

The expected result following a test of the EDI Message Validation filter with a message that fails validation is shown below.



The test message failed validation because it included two fields containing content that did not match the definition's Validation Tables. These fields are identified in the **Message Errors** panel on the output side of the dialog.

Testing the Route with an Invalid Message

Complete the following steps to display the results of a message validation failure in the Management Console.

1. 1

Ensure all components in the **EDI Message Definition** route, and the route itself, are checked in and started.

2. 2

Place a copy of the *testMessage - Validation Error.edi* file into the **Input-Output Directory** communication point's input directory.

3. 3

Note that the test message will be picked up by the route but a validated copy does not appear in the communication point's output directory.

4. 4

Open the Management Console followed by the **Monitoring > Error Queue** screen.

5. 5

Select the most recently added file to open its **Message View** screen.

Model Answer

When the *testMessage - Validation Error.edi* file fails validation, it will be sent to the Management Console's **Error Queue**. Opening its **Message View** followed by the EDI Message Validation filter's **Event Detail** screen reveals the reason for the validation failure. This should be the same as when the filter was independently tested.

Message View 1.26658865397888.57

Displaying message 1 of 1 from the result set named 'Error Queue' [Back to Result Set](#)

Reinject Redirect Reprocess Edit Delete

Message Events Always Show Outputs Last refreshed at 13:04

COMPONENT	TIMESTAMP	DESCRIPTION
Input-Output Directory	Apr 19, 2021 13:03:50.581	Received communication point
Test Route	Apr 19, 2021 13:03:50.590	Placed
EDI Message Validation	Apr 19, 2021 13:03:50.594	Filtering succeeded, took 2.694 ms
EDI Message Validation	Apr 19, 2021 13:03:50.594	Filtering error
Error Queue	Apr 19, 2021 13:03:50.594	

Event Detail

EDI Message Validation fails with 3 validation error(s).
Error 1001 Required segment missing EVN
Error 1008 Field too long: 4 > 3 PV1/HospitalService
Error 1014 Field value not found in validation table
PV1/HospitalService

Message Properties

Property Name	Property Value
BaseFilename	testMessage - Validation Error
directory:BodySize	390
FileCreationTime	1618794225684
FullFilename	testMessage - Validation Error.edi
InputTime	1618794230581
rhapsody:InputCommunicationPoint	Input-Output Directory
rhapsody:InputCommunicationPointId	
Suffix	.edi
ValidationErrorCount	3
ValidationResult	invalid

Showing 10 of 10

Message Body

Tree View Flat Wire

Show Whitespace Encoding: windows-1252 Size: 390 bytes

```
MSH|^~\&|Rhapsody|Rhapsody|CDR|Test|20090917105057||ADT^A03|CA0313|P|2.4\r
PID||84568-4564^^^MRN^MRN~104532R^^MRN^MRN||Cardinal^John^Q||19491112023046|M\r
PD1||Bough Family Clinic^1101|Gx10-000998^WOOD^Brandon^R^D^r^Q^S\r
PV1|I|QS-H1^Bd0038^F1003|I||Qx10-000424^Sanson^Elizabeth^T^D^r^Q^S||CARD|||7|||E84568-4564.3|
```

Download

Note the **ValidationErrorCount** and **ValidationResult** properties. These were added to the message by the validation filter. You could use these to filter invalid messages in your configuration.

Exercise - Hold Queue

The Hold Queue filter provides the ability to send messages to the **Hold Queue** for review and further action. It is commonly used in the development process to temporarily halt the flow of messages along a route so they can be inspected to see, for example, the results of processing by the previous component. Messages on the **Hold Queue** can be viewed, edited and selectively released in the Management Console.

The Hold Queue filter has no configurable properties.

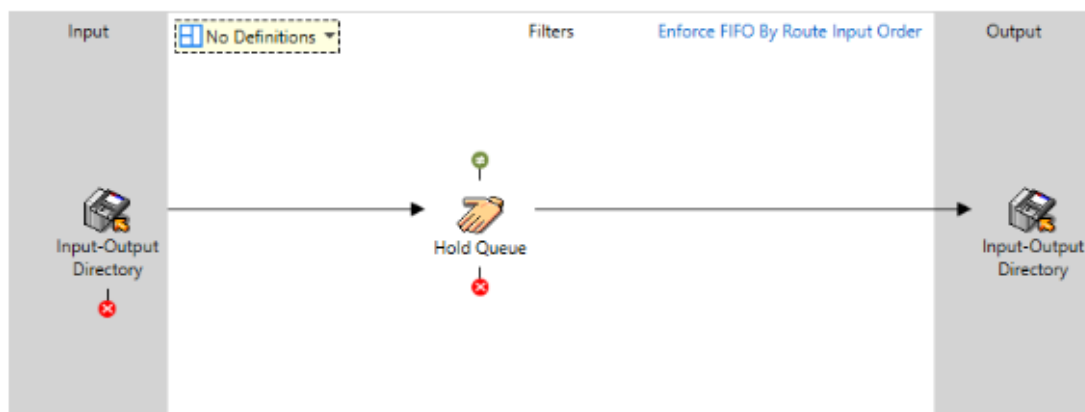
Searching for messages on the hold queue

Messages on the **Hold Queue** will not be included in the results of a search that includes communication points in Output mode. This is because messages sent to the **Hold Queue** do not go through an output communication point to get there.

When to use the Hold Filter

While the Hold Queue filter can be used when testing a solution, it should never be used as a permanent component in a production route. This is because messages must be manually released from the queue, which means it is possible for them to be overlooked and reach unacceptably high numbers. Queued messages will also reduce the available disk space on the Rhapsody server.

Exercise



Configure the route

Create the above route by repeating a modified version of the steps you used to create the **HL7 ACK Generation** route earlier in this module.

Configure the filter

Complete the following steps to test the action of the Hold Queue filter:

1. 1

Add the Hold Queue filter to the route then check in all changes. Start the route and its two communication points.

2. 2

Place a copy of the *testMessage.edi* file into the Directory communication point's Input directory.

3. 3

Open the Monitoring Console and note the following.

- The message has not reached the Output communication point.
- In the **Monitoring** menu, a counter next to the **Hold Queue** entry indicates there is at least one message on this queue.
- Open the **Hold Queue**. The most recently added message has a timestamp that identifies it as the one you added in step 2 above.

Hold Queue Completed Reload No saved searches

Timeframe From / To Show Last From To

Search Order ↑ ↓ Search Options Hide

Routes Any Component ×

Search Reset Quick Filter

☒	TIME	COMPONENT	ROUTE	NEXT DESTINATION	+
☒	Jun 19, 2019 15:51:24.089	Message Input	Hold Messages	Hold ADT Messages	

1 message(s) found (1 selected)

Reinject Redirect Release Delete Download Export Delete All Download All Export All

The following options are available for messages you have selected on the **Hold Queue**; experiment to test their actions:

- **Reinject**
Return the message to the communication point on which it arrived on the route; in this case the Directory communication point.
- **Redirect**
Move the message to any selected component in the route to continue processing.
- **Release**
Release the message so that it can move to the next component in the route; in this case, the Sink communication point.
- **Delete (Delete All)**
Remove copies of all or selected messages from the Rhapsody engine.
- **Download (Download All)**
Download a copy of all or selected messages to a specified location.
- **Export (Export All)**
Export a copy of all or selected messages to a specified location. Selecting this option exports not only the selected messages but their associated metadata as well.
- *Lesson 14 of 15*

• **Exercise – JavaScript (v.2)**

- This exercise introduces the JavaScript (v.2) filter. There are no steps you need to follow; it is just informational.
- The JavaScript filter provides extensive support for manipulating messages and their content, ranging from extracting values to updating content and completely replacing the message. The JavaScript implementation conforms with the ECMAScript standard and includes several additions to access information in the messages, message properties and log files.
- The JavaScript filter accepts a read-only message object as the input object and generates a writable message object as the output object. Typically, a single output message is generated. However, multiple output message objects may be generated (see the Append function: `output.append()` which creates a new output message).
- **Common JavaScript Filter Actions**
 - Below are a number of common JavaScript expressions for extracting or setting parts of a message body.

```
// Extract a field value from a specified message path
```

```
var fieldValue = input[0].getField(path_to_field);
```

```
// Extract a field value from a specified message path
```

```
var fieldValue = input[0].getField(path_to_field);
```

```
// Update a field value
```

```
next.setField(path_to_field, new_field_value);
```

```
// Extract the complete message body
```

```
var body = next.text;
```

```
// Completely replace the message body
```

```
next.text = body;
```

When the message has a structure recognized by the engine, the path to a field can be specified in a XPath-like notation using / or . for messages with an associated message definition (for example MSH.MessageType.MessageType), or XPath notation for XML messages.

JavaScript Properties

The **Configuration** tab has an editing panel where you can enter the code. It also provides boilerplate examples of common tasks. This boilerplate code should be deleted as you configure the filter.

Shared JavaScript Libraries

Rhapsody includes a **Shared JavaScript Libraries** manager, which allows you to add, edit or delete shared JavaScript libraries and functions stored on the Rhapsody engine.

① The configuration and use of the JavaScript (v.2) filter and its shared libraries are described in greater detail in the *Rhapsody Professional* course.

